

Cosmos Policy 代码导读

本文档帮助研究者快速理解 Cosmos Policy 项目的代码架构、核心原理和关键实现。基于论文 *"Cosmos Policy: A Vision-Language Foundation Model for Robot Control"* 和代码库分析。

1. 项目概览

Cosmos Policy 是 NVIDIA 出品的视觉-语言基础模型（VLM），用于机器人控制。其核心思想是：**将机器人策略（action generation）、世界模型（world modeling）和价值函数（value prediction）统一在一个视频扩散模型中。**

核心创新

- 把 action、future state、value 全部"伪装"成视频帧，编码进统一的 latent space
- 基于 NVIDIA Cosmos Predict2 视频生成模型进行 fine-tune
- 支持 Best-of-N planning：生成多个 action 候选，用 value function 选最优

支持的三个平台

平台	Action 维度	Proprio 维度	Action Chunk 长度
LIBERO	7	9	16
RoboCasa	7	9	32
ALOHA	14	14	50

2. 目录结构

```

cosmos_policy/
├── _src/
│   ├── imagineire/
│   │   ├── attention/
│   │   ├── modules/
│   │   └── utils/
│   ├── predict2/
│   │   ├── models/
│   │   ├── networks/
│   │   ├── tokenizers/
│   │   ├── conditioner.py
│   │   ├── text_encoders/
│   │   └── configs/
│   └── reason1/
├── models/
│   ├── policy_text2world_model.py
│   └── policy_video2world_model.py
├── modules/
│   ├── cosmos_sampler.py
│   └── hybrid_edm_sde.py
├── conditioner.py
├── constants.py
├── trainer.py
├── config/
│   ├── config_v2.py
│   ├── config.py
│   ├── defaults/
│   ├── conditioner/
│   └── experiment/
│       └── cosmos_policy_experiment_configs.py
├── datasets/
│   ├── libero_dataset.py
│   ├── robocasa_dataset.py
│   ├── aloha_dataset.py
│   ├── dataset_common.py
│   ├── dataset_utils.py
│   └── t5_embedding_utils.py
├── experiments/
│   └── robot/
│       ├── cosmos_utils.py
│       ├── robot_utils.py
│       ├── libero/
│       ├── robocasa/
│       └── aloha/
├── scripts/
│   └── train.py
├── tokenizers/

```

内部依赖 (不直接修改)

NVIDIA Imagineire 训练框架

Flash Attention 2/3, cuDNN 注意力后端

基础模块: EDM SDE, Sampler, VAE

分布式、checkpoint、logging 工具

Cosmos Predict2 视频生成模型

Text2World, Video2World 基础模型

DiT 网络架构 (minimal_v4_dit.py)

Cosmos VAE tokenizer, WAN 2.1 tokenizer

GeneralConditioner (T5 + CLIP)

T5 text encoder

Predict2 默认配置

Reason1 模型 (VLM 推理, Qwen-based)

★ Cosmos Policy 核心模型

CosmosPolicyDiffusionModel (基础扩散模型)

CosmosPolicyVideo2WorldModel (视频条件扩散模型)

★ Policy 扩展模块

扩展的采样器

混合 sigma 分布 (70% log-normal + 30% uniform)

★ 可变 Condition 数据类

平台常量 (自动检测)

CosmosPolicyTrainer (epoch 跟踪)

★ 配置系统

ConfigV2 主配置

推理配置入口

默认 model、tokenizer 注册

Conditioner 配置

★ 实验配置 (每个 benchmark 一个)

★ 数据集实现

LIBERO 数据集

RoboCasa 数据集

ALOHA 数据集

共用数据逻辑

数据增强、JPEG 压缩

T5 文本嵌入预处理

★ 评估脚本

模型加载、action 生成、value 预测

通用评估工具

LIBERO 评估

RoboCasa 评估

ALOHA 部署

训练入口脚本

Policy 专用 tokenizer 封装

```
└─ utils/                # Checkpoint 加载等工具
└─ verify_installation.py # 安装验证
```

3. 架构设计：核心思想

3.1 Latent Sequence 设计 — 万物皆视频帧

这是整个项目最核心的设计思想。模型把所有输入输出统一编码为一个 **latent 视频序列**：

```
LIBERO (state_t=9):
[blank, curr_proprio, curr_wrist_img, curr_primary_img, action,
 future_proprio, future_wrist_img, future_primary_img, value]
  0         1         2         3         4         5         6         7
8

RoboCasa/ALOHA (state_t=11):
[blank, curr_proprio, curr_wrist1, curr_wrist2/primary2, curr_primary,
 action, future_proprio, future_wrist1, future_wrist2/primary2, future_primary, value]
  0         1         2         3         4         5         6         7
8         9        10
```

关键代码： `cosmos_utils.py:61-100` 的 `get_latent_indices_from_model_config()` 根据配置返回各帧的索引位置。

3.2 Latent Injection — 如何把 action/value "塞进" 视频帧

Action 和 value 本质上是低维向量（如 7 维 action），但视频帧的 latent 是高维张量（如 $16 \times 28 \times 28$ ）。解决方法是将低维向量重复填充到整个 latent 帧：

```
# policy_text2world_model.py:45-109
def replace_latent_with_action_chunk(x0, action_chunk, action_indices):
    # action_chunk: (B, chunk_size, action_dim) → flatten → repeat → fill latent
    flat_action = action_chunk.reshape(batch_size, -1)
    repeated_action = flat_action.repeat(1, num_repeats)[: , :latent_elements]
    new_x0[batch_indices, :, action_indices, :, :] = repeated_action.reshape(...)
```

同理， `replace_latent_with_proprio()` 将本体感觉状态注入 latent 帧。

3.3 Condition Frame vs Denoise Frame

- **Condition frames**（条件帧）：已知观测（当前 proprio、当前图像），通过 `sigma_conditional=0.0` 设为无噪声，直接注入
- **Denoise frames**（去噪帧）：需要预测的部分（action、future state、value），从纯噪声开始逐步去噪

```
config 中:
min_num_conditional_frames = 4 # blank + 3 个观测帧 (LIBERO)
state_t = 9                  # 总共 9 帧
```

条件帧的注入策略是 `frame_replace`：在每步去噪时，将条件帧替换回 ground truth latent。

4. 模型继承体系

```
Text2WorldModel (Predict2 基础模型)
├── CosmosPolicyDiffusionModel (policy_text2world_model.py)
│   ├── 新增: compute_loss() - 多任务 loss 计算
│   ├── 新增: generate_samples_from_batch() - 带方差缩放的采样
│   ├── 新增: replace_latent_with_action_chunk/proprio
│   └── 新增: 训练/推理 data batch 构建
└── CosmosPolicyVideo2WorldModel (policy_video2world_model.py)
    ├── 新增: 视频帧条件注入
    ├── 新增: 高 sigma 策略
    └── 新增: FlowUnIPC scheduler 支持
```

CosmosPolicyDiffusionModel 关键方法

方法	位置	作用
<code>compute_loss()</code>	<code>policy_text2world_model.py</code>	EDM loss 计算，支持多种 loss mask
<code>generate_samples_from_batch()</code>	<code>policy_text2world_model.py</code>	扩散采样生成，支持 variance scale
<code>get_data_and_condition()</code>	<code>policy_text2world_model.py</code>	构建 data batch 和 condition
<code>get_x0_fn_from_batch()</code>	继承	构建 denoiser 函数

三类 loss mask (`compute_loss` 中)

- `mask_loss_for_action_future_state_prediction`：Demo 样本只计算 action 和 future state 的 loss
- `mask_current_state_action_for_value_prediction`：Value 预测时遮住当前状态和 action
- `mask_future_state_for_qvalue_prediction`：Q-value 预测时遮住 future state
- `action_loss_multiplier`：对 action loss 加权

5. 训练流程

5.1 入口

```
torchrun --nproc_per_node=1 -m cosmos_policy.scripts.train \
    --config=cosmos_predict2_2b_480p_libero
```

5.2 训练脚本 (`scripts/train.py`)

1. 分布式初始化 (`distributed_init`)
2. 加载配置 (`load_config` → `ConfigV2`)
3. 实例化模型 (`instantiate(config.model)` → `CosmosPolicyVideo2WorldModel`)
4. 手动创建 `DistributedSampler` (避免重复数据集创建)
5. 创建 `DataLoader`
6. `trainer.train(model, dataloader_train, dataloader_val)`

5.3 Trainer (`trainer.py`)

`CosmosPolicyTrainer` 继承 `ImaginaireTrainer`, 核心改动:

- 添加 **epoch 跟踪** (`sampler.set_epoch(epoch)`)
- 主训练循环: `while True` → 遍历 `dataloader` → `training_step()` → 梯度累积 → `optimizer step` → `checkpoint`

5.4 Loss 计算 (`compute_loss`)

训练数据有三种来源, 对应不同的 `loss mask`:

数据类型	<code>rollout_data_mask</code>	计算的 <code>loss</code>
Demo 数据	0	action + future state + value (辅助)
World model rollout	1, <code>world_model=1</code>	future state (主) + value (辅助)
Value function rollout	1, <code>value_function=1</code>	value (主)

Loss 公式: **EDM weighted MSE loss**

```
edm_loss = MSE(x0, model_pred.x0) × weight_per_sigma
```

5.5 学习率调度

```
# LIBERO 配置
cycle_lengths = [30000, ∞]           # 30K decay + constant
warm_up_steps = [1000, 0]
f_max = [1.0, 0.06]                 # 从 1e-6 warmup 到 1.0, 然后衰减到 0.06
```

6. 推理流程

6.1 整体管线

```

观测 (images, proprio, task_description)
↓
构建 data_batch (latent injection)
↓
get_action_prediction() → 扩散去噪 → 解码 action latent
↓
[可选] get_future_state_prediction() → 自回归预测未来状态
↓
[可选] get_value_prediction() → 预测 value
↓
Best-of-N: 选 value 最高的 action
↓
执行 action, 滑窗推进

```

6.2 Action 生成 (`cosmos_utils.py`)

```

def get_action_prediction(model, data_batch, dataset_stats, observation, ...):
    # 1. 构建条件帧 (proprio, images → latent)
    # 2. 随机初始化 action latent frame
    # 3. model.generate_samples_from_batch() 去噪
    # 4. 从 latent 中提取 action
    # 5. 反归一化 action
    # 6. 返回 action chunk

```

6.3 Best-of-N Planning

```

# run_libero_eval.py 中的 run_episode():
for query_idx in range(num_queries_best_of_n): # 并行生成 N 个 action
    action_return_dict = get_action_prediction(...)
    if ar_value_prediction:
        value_return_dict = get_value_prediction(...)

# 选 value 最高的 action
best_seed = max(seed_to_return_dict, key=lambda x: x[1][2])
action_queue.extend(best_actions)

```

6.4 并行推理

支持多 GPU 并行 Best-of-N:

- `use_parallel_inference=True`
- `WorkerPoolManager` 在多 GPU 上各加载一份模型
- 每个 worker 独立推理, 汇总结果

7. 配置系统

7.1 配置继承链

```
ConfigV2 (config_v2.py)
└─ 注册 defaults 列表:
    optimizer=fusedadamw
    scheduler=lambdalinear
    model=policy_fsdp
    conditioner=video_prediction_conditioner
    tokenizer=policy_wan2pt1_tokenizer
    experiment=<具体实验>

└─ 实验配置 (cosmos_policy_experiment_configs.py):
    cosmos_predict2_2b_480p_libero          # LIBERO 训练
    cosmos_predict2_2b_480p_libero__inference_only # LIBERO 推理
    cosmos_predict2_2b_480p_roboCasa_*      # RoboCasa
    cosmos_predict2_2b_480p_aloha_*         # ALOHA
```

7.2 关键配置项

```
model.config:
    state_t = 9 # LIBERO: 9 帧; RoboCasa/ALOHA: 11 帧
    min_num_conditional_frames = 4 # 条件帧数
    sigma_conditional = 0.0 # 条件帧无噪声
    conditioning_strategy = "frame_replace"
    chunk_duration = 33 # WAN 2.1 tokenizer: 1 blank + 32 images (8帧×4 duplicates)
    resize_online = True
    resolution = "224"

sde:
    hybrid_sigma_distribution = True # 70% log-normal + 30% uniform
    sigma_max = 200 (训练) / 80 (推理)
    sigma_min = 0.01 (训练) / 4 (推理)
```

7.3 平台差异

参数	LIBERO	RoboCasa	ALOHA
state_t	9	11	11
conditional_frames	4	5	5
chunk_duration	33	41	41
chunk_size	16	32	50
额外视角	wrist	wrist + secondary	left_wrist + right_wrist

8. 数据集

8.1 数据集类

每个 benchmark 有自己的 Dataset 类，继承自 `dataset_common.py`：

- `LIBERODataset` (`libero_dataset.py`)
- `RoboCasaDataset` (`robocasa_dataset.py`)
- `ALOHADataset` (`aloha_dataset.py`)

8.2 数据采样策略

每个 batch 混合三种数据来源：

1. **Demo 数据** (成功示范)： `demonstration_sampling_prob=0.5`
2. **成功 Rollout**： `success_rollout_sampling_prob=0.5`

每个样本包含：(current_state, action, next_state, value_label)

8.3 Value Label 计算

```
# 使用 discounted return 作为 value label
return_value_function_returns = True
gamma = 0.99 # LIBERO/RoboCasa
gamma = 0.998 # ALOHA (episode 更长)
```

8.4 数据增强

- 图像增强 (`use_stronger_image_aug=True`)
- JPEG 压缩增强
- Proprio 和 action 归一化 (`dataset_stats` 提供 mean/std)
- T5 文本嵌入预计算 (离线保存为 `.pkl`)

8.5 WAN 2.1 Tokenizer 的 "4x 重复"

WAN 2.1 tokenizer 的时间压缩因子为 4，即 4 帧图像编码为 1 个 latent 帧。因此每个概念（如 "action"）需要 4 张相同图像输入，产生 1 个 latent 帧：

```
chunk_duration = 33 = 1 blank + 8 concepts × 4 duplicates
                    (proprio, wrist, primary, action, future_proprio, future_wrist, future_primary,
value)
```

9. 扩散采样模块

9.1 HybridEDMSDE (`modules/hybrid_edm_sde.py`)

训练时的 sigma (噪声水平) 采样：

- **70%** 从 log-normal 分布采样 (标准 EDM 方式)

- **30%** 从 `uniform(1.0, 85.0)` 采样（增加高噪声样本权重）

```
distribution_choice = torch.rand(batch_size) < 0.7 # 70/30 split
```

9.2 CosmosPolicySampler (`modules/cosmos_sampler.py`)

扩展基础 Sampler:

- `sample_clean=True` 时，额外执行一次 denoiser 得到完全去噪结果
- 支持 `num_steps=1` 的特殊情况（直接一步去噪）

10. 关键文件速查表

你了解...	去看...
模型架构整体设计	<code>models/policy_text2world_model.py</code>
视频条件注入	<code>models/policy_video2world_model.py</code>
Action 如何塞进 latent	<code>policy_text2world_model.py:45-109</code>
Latent 序列索引	<code>experiments/robot/cosmos_utils.py:61-100</code>
Loss 计算 (含 mask)	<code>policy_text2world_model.py</code> 的 <code>compute_loss()</code>
训练入口	<code>scripts/train.py</code>
训练循环	<code>trainer.py</code>
推理/评估入口	<code>experiments/robot/libero/run_libero_eval.py</code>
Action 生成	<code>experiments/robot/cosmos_utils.py</code> 的 <code>get_action_prediction()</code>
Best-of-N planning	<code>run_libero_eval.py</code> 的 <code>run_episode()</code>
实验配置 (超参数)	<code>config/experiment/cosmos_policy_experiment_configs.py</code>
平台常量	<code>constants.py</code>
数据集实现	<code>datasets/libero_dataset.py</code> 等
Sigma 分布	<code>modules/hybrid_edm_sde.py</code>
采样器	<code>modules/cosmos_sampler.py</code>
Conditioner	<code>conditioner.py</code>
配置系统	<code>config/config_v2.py</code>
DiT 网络架构	<code>_src/predict2/networks/minimal_v4_dit.py</code>
Tokenizer	<code>_src/predict2/tokenizers/cosmos.py</code> / <code>wan2pt1.py</code>

11. 作为 Baseline 做研究的关键理解

11.1 可修改的切入点

1. 新增 **benchmark**:

- 新建 `datasets/your_dataset.py`
- 在 `constants.py` 中添加平台常量
- 在 `cosmos_policy_experiment_configs.py` 中添加实验配置
- 修改 `get_latent_indices_from_model_config()` 添加新的 latent 序列布局

2. 修改 **action** 预测方式:

- 修改 `replace_latent_with_action_chunk()` 中的 latent injection 逻辑
- 修改 `get_action_prediction()` 中的 action 解码逻辑

3. 修改 **planning** 策略:

- 修改 `run_episode()` 中的 Best-of-N 逻辑
- 修改 value function 的使用方式

4. 修改模型架构:

- 继承 `CosmosPolicyVideo2WorldModel` 添加新功能
- 修改 `compute_loss()` 改变训练目标

11.2 运行环境要求

- 必须 **CUDA GPU** (代码中硬编码 `cuda:0`)
- Flash Attention 2 或 3
- cuDNN
- 推荐 RTX 3060 12GB+ (推理), RTX 3080 10GB+ (推理 + planning)
- Python 3.11, CUDA 12.8.1, Ubuntu 24.04

11.3 运行命令

```
# 训练
torchrun --nproc_per_node=N -m cosmos_policy.scripts.train \
    --config=cosmos_predict2_2b_480p_libero

# 推理 (LIBERO)
uv run -m cosmos_policy.experiments.robot.libero.run_libero_eval \
    --config cosmos_predict2_2b_480p_libero__inference_only \
    --ckpt_path nvidia/Cosmos-Policy-LIBERO-Predict2-2B \
    --config_file cosmos_policy/config/config.py \
    --task_suite_name libero_10 \
    --seed 195
```